

Document Number: P2217R0
Date: 2020-08-29
Audience: SG16
Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [June 10th, 2020](#)
- [June 17th, 2020](#)
- [July 8th, 2020](#)
- [July 22nd, 2020](#)
- [August 12th, 2020](#)
- [August 26th, 2020](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)

June 10th, 2020

Draft agenda:

- Discuss terminology updates to strive for in C++23
 - [P1859R0: Standard terminology character sets and encodings](#).
 - Establish priorities for terms to address.
 - Establish a methodology for drafting wording updates.

Attendees:

- Alisdair Meredith
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Marcos Bento

- Mark Zeren
- Martinho Fernandes
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- A round of introductions was held for the benefit of new attendees.
- Zach asked for everyone to contribute to the Boost.Text review scheduled to start on the following day, June 13th.
 - Contributors will need to subscribe to the boost@lists.boost.org mailing list at <https://lists.boost.org/mailman/listinfo.cgi/boost>.
 - An introductory invitation for SG16 members was posted to the SG16 mailing list and is available at <https://lists.isocpp.org/sg16/2020/06/1499.php>.
- Tom mentioned that work has progressed on establishing a shared calendar for all WG21 telecons. Official announcements are expected soon. For now, BlueJeans calendar invites will continue to be sent as usual, but may be discontinued in the future if the shared calendar works well for everyone.
- Discuss terminology updates to strive for in C++23
 - Tom introduced the topic.
 - Per prior meetings, modernizing terminology in the standard is an SG16 goal for C++23.
 - Tom expressed uncertainty with regard to the best starting point for discussion, but suggested starting by reviewing a set of existing terms used in the standard that he included in an [email to the SG16 mailing list](#) right before the meeting.
 - Corentin expressed desire to take a holistic approach to updating the wording and directed attention to his [D2178R0 draft attached to a message sent to the SG16 mailing list](#).
 - Corentin suggested splitting the effort to focus first on core wording, then on library wording.
 - PBrett opined that core wording will be difficult and would prefer a single paper to address it, but potentially multiple papers to address library wording.
 - PBrett noted that some library components treat non-text as text. For example, file names, command line arguments, stream contents, and environment variables.
 - Hubert suggested inserting a third phase up front to just establish terminology itself.
 - Alisdair agreed noting that commonly understood terminology provides the tools necessary to discuss wording.
 - Steve expressed a desire to introduce new terms in order to facilitate easier communication; specifically new short terms that can substitute for otherwise wordy phrasing.
 - Steve stated that we'll need to re-word with expectation of impact to existing implementations.
 - Tom agreed noting that he ran into such situations drafting [P2029](#). This happens due to interaction with core issues and discovery of existing conformance issues in implementations.
 - Corentin replied that any such impact should be minimal, and should effectively be bug fixes, each of which has limited impact to existing implementations.
 - PBrett asked if we have general agreement for splitting the work in three phases as indicated.
 - No objections were raised.
 - Hubert stated that we may need to introduce new terms.
 - Tom suggested that, perhaps, we should start discussion with *character* first.
 - Hubert responded that [P1859R0](#) already discussed *abstract character* and no one raised concerns.

- Discussion turned to the first item in the list of terms Tom sent to the mailing list, "The encoding of source files".
- Someone noted that the source may not be a file, or even a digital resource with an encoding in any traditional sense.
- Steve responded that Richard Smith is a conforming implementation of the standard.
- Alisdair asked if the standard should rule out source files contained in .zip files.
- Tom replied that he wasn't aware of such translation phase 1 abilities being challenged and that any proposed changes should strive to preserve such abilities.
- Corentin observed that, if the source input is an image, there is no traditional character encoding or character set, but a stream of characters is still available.
- Hubert suggested that it may be useful to introduce the notion of a logical source file that is distinct from any physical representation.
- Steve noted that a path through that logical representation is currently required to retrieve original spelling of characters in raw string literals.
- Corentin opined that the current machinery works and that it is nice to be able to discard the notion of a physical source representation after phase 1.
- Hubert stated that translation phase 1 does too much for one phase right now.
- Corentin agreed and stated a preference that translation phase 1 only perform character mapping.
- Jens described how translation phase 1 could be divided into sub-phases. Phase 1A would produce logical characters and phase 1B would map to *universal-character-names*.
- Jens opined that the notion of physical source file is too limiting; other input forms should not be excluded.
- Corentin reiterated his fondness for discarding physical details after translation phase 1.
- Jens stated that the current method of reverting portions of translation phases 1 and 2 to retrieve the original spelling for raw string literals is very hacky; it would be better to preserve the original information in a more direct manner.
- Tom asked if there are additional benefits that could be had by addressing the raw string literal issue.
- Alisdair responded that, since trigraphs were removed, this scenario is now the tail wagging the dog.
- Steve noted that addressing it could solve the [issue recently discussed on the SG16 mailing list](#) involving EBCDIC characters that get converted to *universal-character-names* that are not semantically preserving.
- Hubert noted that we still have outstanding issues with raw string literals and new line characters.
- Corentin suggested that introduction of an additional character mapping may be heading in the wrong direction; we want to make things simpler and being able to focus solely on Unicode post translation phase 1 would help that goal.
- Hubert responded that there is a benefit to having the standard reflect the general case.
- Tom suggested it would be useful to give this concern a name and move on to other discussion.
- Alisdair raised the relationships between character, character set, and character encoding.
- Hubert pondered whether we need character repertoire and noted over use of the term character set where character encoding is often meant.
- PBrett suggested discontinuing the use of character set.
- Corentin disagreed noting that the execution character set is a character set and that discussion of code points requires a character set as opposed to a repertoire.
- PBrett asked why a character repertoire plus an encoding doesn't suffice.
- Corentin responded that his explanation was based on Unicode definitions.
- Hubert stated that use of the Unicode definitions is fine for discussion purposes; the basic execution character set is sometimes used where an encoding is intended unless you subscribe to the belief that `wchar_t` implies a trivial encoding.

- Hubert continued noting that the basic execution character set is sometimes used as a repertoire, and at other times used as a character set.
- Tom responded that he thinks of the basic execution character set as defining a restriction on character sets since it places some constraints on code assignments; the code points for digits 0-9 must be in sequence, and the code point value for NUL must be 0.
- Hubert noted that the abstract numeric values mapped to abstract characters are sometimes fictitious.
- Corentin discussed the idea of the internal character set being a repertoire; that works up until translation phase 5 when conversions for literals produce objects with values.
- Tom provided a description of his understanding of character repertoire, character set, and character encoding. A character repertoire is a set of abstract characters. A character set is a map of abstract characters corresponding to some character repertoire to numeric code point values. A character encoding is a specification for how to encode those numeric code point values as a sequence of code units.
- Tom asked if any of those definitions were surprising.
- PBrett expressed a little surprise with regard to the implied need for a character encoding to have an associated character set since an encoding could specify how to encode abstract characters directly.
- Steve stated that, according to Unicode, a coded character set defines a map of characters to numeric code point values, but that a character set in general need not specify such mappings.
- Tom asked for confirmation that we should prefer the term coded character set when we explicitly mean a map of characters from a repertoire to numeric code point values.
- Steve responded, yes.
- PBrett observed that, for ISO/IEC 8859 specifications other than ISO/IEC 8859-1, the specified character repertoire is a subset of the Unicode character repertoire, but the specified character set is not a subset of the Unicode character set since code point assignments differ for some non-ASCII cases.
- PBrett also observed that the basic source character set is a repertoire, but the compiler must also define an associated coded character set.
- Jens responded that that is true from an implementation perspective, but not with regard to how the standard uses it since the standard permits symbolic evaluation.
- Hubert noted that the standard may not be very consistent in how the existing terms are used, but the use of terms with fewer requirements is useful.
- Hubert expressed concern regarding focus on coded character sets because it isn't clear that abstract numeric code point values are helpful from a specification standpoint.
- Jens responded that it is convenient to be able to discuss a character having a numeric value, but agreed that it is not germane to the standard.
- Jens continued stating that, at the end of the day, we need to encode bytes for a character that was previously abstract; if the use of the character set term is confusing, we can replace it, but that seems like an editorial concern, albeit a useful one to avoid confusion or reduce baggage.
- PBrett expressed support for a new term since character set is often confused with encoding.
- Corentin provided the historical perspective that most legacy character encodings were trivial encodings of code points from a given coded character set, so the terms were almost always interchangeable prior to Unicode.
- Steve stated that numeric code point values for basic source characters are not observable though, per [\[cpp.cond\]p12](#), different values corresponding to them may be observed at different phases of translation.
- Hubert observed that, within the standard, discussion of character sets usually corresponds to the Unicode definition of character encoding schemes.

- Tom summarized; it sounds like we likely have need for character repertoire and character encoding scheme, but perhaps not for character set or coded character set.
- Hubert responded that there may be a need for character set specifically when referring to Unicode.
- Tom pondered whether a coded character set is needed for character literals. The current constraint for the value of a character literal [*Editor's note: other than for multicharacter literals or literals with no representation in the execution character set.*] is that the abstract character can be encoded in a single code unit.
- Jens stated that the only observable character values are code units in Unicode parlance.
- PBrett asked whether *unicode-character-names* fits that picture.
- Jens replied that we do associate them with Unicode code points, but from a standard perspective, they are basically text.
- Hubert suggested use of generalized terminology for these low level concerns with Unicode terminology reserved specifically for Unicode encodings.
- Alisdair noted that encoding matters can't assume octets.
- Hubert agreed, but noted that some of the ISO blessed specifications specify octets and provided a source in chat:
 - "(Source: RFC1866) A function whose domain is the set of sequences of octets, and whose range is the set of sequences of characters from a character repertoire; that is, a sequence of octets and a character encoding scheme determining a sequence of characters."
 - [ISO/IEC 15445:2000](#), 4.3
- Tom suggested we move on to some polls.
- **Poll: We should move forward in three phases. 1) define terminology, 2) address core wording, 3) address library use of terms**
 - **Attendees: 12**
 - **No objection to unanimous consent.**
- **Poll: This group generally believes that C++ lexing and parsing behavior through translation phase 4 can be defined in terms of character repertoires and without the need for coded character values.**
 - **Attendees: 12**
 - **S F F N A S A**
3 8 0 0 1
 - SA: We'll have the issue that we cannot preserve byte values from the source stream; this loses the relation to bytes and is overly abstract.
 - [*Editor's note: After the telecon, Hubert [posted to the SG16 mailing list](#) to express agreement with the SA position: "I agree ... that the strict use of abstract characters introduces problems where a coded character set contains multiple values for a single abstract character/contains characters that are canonically the same but assigned different values."]*
- Tom discussed options for scheduling the next SG16 telecon noting that he will not be available the week of June 22nd which would be the next time we would meet following our usual cadence. The group agreed to meet in one week, on June 17th, in order to maintain momentum on this topic.

June 17th, 2020

Draft agenda:

- Continue discussion of terminology updates to strive for in C++23

- Resume discussion of relationships between (abstract) character, (character) repertoire, (coded) character set, and character encoding.
 - Review ISO/IEC 10646:2017 section 3 terms and definitions
 - https://standards.iso.org/ittf/PubliclyAvailableStandards/c069119_ISO_IEC_10646_2017.zip
 - Review Unicode section 3.4 terms for characters and encodings
 - <https://www.unicode.org/versions/Unicode13.0.0/ch03.pdf>
 - Review the Unicode glossary
 - <https://www.unicode.org/glossary>
 - Review Corentin's email
 - <https://lists.isocpp.org/sg16/2020/06/1493.php>
 - Compare and contrast terms as described by the above resources.
- Determine suitability of ISO/IEC 10646 terms for use in the C++ standard.
- Discuss the relationship of the above terms to named entities in the standard.
- Identify possible terms to add to [\[intro.defs\]](#).

Attendees:

- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Marcos Bento
- Mark Zeren
- Martinho Fernandes
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann

Meeting summary:

- Tom introduced the topic:
 - The intent is continuation of discussion from the prior telecon.
 - Polls taken during the prior telecon were presented and it was noted that mailing list discussions following the telecon may have changed opinions.
- Jens opined that we need more than just a glossary; the mailing list discussion raised examples of characters that can not go through translation phase 1 without information loss. This means we cannot convert to Unicode universally without loss.
- Tom raised a question that Corentin had asked him during private discussion following the telecon. Corentin had asked if, given a string literal and a raw string literal where both are specified with the same source input characters (with extended characters but without escape sequences), whether both strings must have the same encoded contents after translation phase 5.
- PBrett responded that some people assert that raw string literals should effectively copy the byte sequence from the source input.
- Corentin disagreed with such an interpretation and noted that conversions are required.
- Tom presented two possible models for the reversion of *universal-character-names* (UCNs) in raw string literals during translation phase 5.
 - The UCN is reverted to the original source input character and that character is then encoded in the appropriate encoding for the kind of string literal.

- The UCN is reverted to the code point denoted by the UCN and that code point is then encoded in the appropriate encoding for the kind of string literal.
- Corentin opined that the reversion can be accomplished via the as-if rule and translation phase 1 and 5 shenanigans.
- Tom asked Jens to comment on Corentin's interpretation of the as-if rule in this context from a core perspective.
- Jens responded that the question is whether a conforming program could observe the difference.
- Tom replied that implementation-defined behavior is unavoidable here, so the standard can't fully define the behavior on its own.
- Corentin stated that, if you have a Unicode character, conversion to Shift-JIS provides a choice of code point values for some characters.
- PBrett noted that the program can distinguish behavior here.
- Corentin replied that the original source file encoding can't be observed.
- Martinho noted that a program can demonstrate the behavior though.
- Jens stated that programmers have expectations of behavior based on their source file encoding; they expect what they write to be carried through.
- Tom asked if it would be conforming for an implementation to, given an 'Å' (U+00C5, LATIN CAPITAL LETTER A WITH RING ABOVE) or 'Å' (U+212B, ANGSTROM SIGN) in the source input, to always translate both to one or the other in the execution character set.
- Corentin replied that, for Unicode input, we can require preservation of code points.
- PBrett asked if the standard currently permits such translation.
- Steve responded that translation phase 1 is so loose that any imaginable conversion is conforming and provided handling of trigraphs as an example.
- Jens agreed and elaborated; translation phase 1 states that physical source files are mapped in an implementation-defined manner and that mapping can include recognizing and mutating string literals.
- Martinho claimed that an implementation can even recognize every source input file as equivalent!
- Jens agreed, but noted that the implementation has to actually define what it does.
- PBrett noted the utility of such lenience; for Shift-JIS we only need implementation-defined behavior on the input side.
- Steve responded that the conversion to execution character set for Shift-JIS could be lossy, but for the Unicode A-with-ring vs Angstrom-sign case, it need not be.
- Martinho observed that, if a UCN isn't explicitly written in the source, the implementation has freedom to handle the conversion however is desired.
- Tom replied that the implementation has such freedom regardless of whether the UCN is explicit due to translation phase 1 leniency.
- Corentin stated that leaving these conversions as implementation-defined for now will allow us to make progress.
- Jens observed that, for a hypothetical future where Unicode code point pass through is required, the implementation-defined steps in between can be removed.
- Mark asked if, in that world, whether raw string literals would still have to revert UCNs.
- Jens responded yes; translation phase 1 could simulate Unicode input.
- Tom observed that recognition of tokens in translation phase 3 depends on UCNs and asked, when a UCN is reverted, what it is reverted to.
- Jens responded that it is reverted to an extended character.
- Tom replied that extended characters are not reflected in the grammar and stated that this has implications for the stringize operator in the case where a macro name spelled with an extended character is stringized.
- PBrett stated that an extended character is any character in the internal character set that is not a member of the basic source character set.
- Corentin stated that the mapping from every extended character to a UCN is required.

- Hubert noted that the internal character set is effectively Unicode and that this differs from the model used for C.
- Jens agreed and observed that the requirement only exists because extended characters must be representable as a UCN.
- PBrett asked if this avoids the need to discuss the Unicode character set.
- Jens responded that that is the status quo; the question is whether we need to carve an exception for extended characters that don't roundtrip through Unicode and whether that is desirable or whether loss of some information is ok.
- Jens noted that the UCN mechanism permits translation through an ASCII only preprocessor.
- Jens summarized; there are two reasonable positions:
 - The status quo; the standard doesn't recognize the existence of characters that don't roundtrip through Unicode, or
 - The standard should be updated to recognize the possibility of such characters and specify behavior for them.
- Corentin agreed with Jens' summary, but noted another possible position, the standard could specify conversion via Unicode, but require semantic preservation for extended characters.
- PBrett asked if the internal character set could be replaced with the Unicode character set since the standard requires it to be isomorphic anyway.
- Jens expressed concerns about doing so since that would require defining behavior for unassigned code points.
- Hubert stated that some implementations map characters to a limited internal character set that only supports the current locale; conversion through Unicode is a complicated process to get a simple result for round tripping.
- Hubert observed that C already adopted a model that doesn't force the internal character set to be Unicode.
- Jens noted that C supports UCNs and asked how its model avoids these issues.
- Hubert referenced the ["C99 rationale" document](#) and explained that it documents three models for handling UCNs. C chose one model and C++ chose another.
- Hubert noted that limitations with regard to eager conversion of extended characters to UCNs in translation phase 1 effectively requiring all extended characters to have representation in Unicode are not discussed in the document.
- PBrett asked if implementations that support extended characters not represented in Unicode would become non-conforming if the internal character set was defined as being Unicode.
- Hubert responded that no, the model adopted for C++ that permits observability of UCNs is defective; it seems that C++ failed to specify the intended behavior.
- [*Editor's note: The referenced ["C99 rationale" document](#), in section 5.2.1, subsection "UCN models", states:*

Once this was adopted, there was still one problem, how to specify UCNs in the Standard. Both the C and C++ committees studied this situation and the available solutions, and drafted three models:

- Convert everything to UCNs in basic source characters as soon as possible, that is, in translation phase 1.
- Use native encodings where possible, UCNs otherwise.
- Convert everything to wide characters as soon as possible using an internal encoding that encompasses the entire source character set and all UCNs.

Furthermore, in any place where a program could tell which model was being used, the standard should try to label those corner cases as undefined behavior.

]

- Jens summarized; the UCN model was chosen by C++ decades ago and it has issues. C chose a different model, and Hubert suggests that use of that model would not require round trip through Unicode

and thus may make more programs well-formed.

- PBrett asked if the C model retains the notion of an internal character set.
- Hubert responded that C's model doesn't introduce UCNs in translation phase 1; rather it has extended characters and wording that achieves the same result. C has explicit wording to handle basic and extended characters.
- Jens asked how C avoids handling UCNs in a character literal.
- Hubert responded that C doesn't have to define the special property of what can be encoded in a character literal.
- Hubert noted that, if we move away from UCNs, it will be necessary to add wording to handle extended characters.
- PBrett stated that it sounds like the C model permits the internal character set to be a super set of Unicode.
- Tom noted that Corentin and Steve have both expressed a preference for translating extended characters to Unicode code points that are maintained distinctly from UCNs.
- Hubert responded that code point is just a term. If we switch models, then we'll need to add wording to handle these scenarios; it might not be less wording than is needed for UCNs.
- Corentin agreed, but noted that it would avoid the need for the UCN reversion that currently happens in raw string literals and stringize operations.
- PBrett asked how the notion of an extended character differs from a code point; code point has an implied character set association, but extended character doesn't.
- Hubert responded that there is a distinction: extended character excludes basic source characters. This distinction may not be useful.
- Jens expressed concern about potentially losing that distinction since extended characters can only appear in a limited number of contexts.
- Corentin expressed a preference for use of common terminology and that extended characters would make it difficult to discuss behavior in Unicode terms.
- Hubert noted that extended characters just provide differentiation from basic source characters because the latter have additional requirements placed on them.
- PBrett observed that code points require correlation with a character set, but that an extended character can have distinct code points in a single character set.
- Steve noted that code point values don't tend to be observable but that code units are.
- Hubert stated that the term code point is probably not correct to describe a character that can apply generically to multiple character sets.
- Steve listed some of the requirements for the members of the basic execution character set; each such character is encoded as a single code unit with a non-negative value, and the code unit values for the digits 0-9 have consecutive values.
- Jens noted that the term "code point" implies an associated numeric value, but that such a value is not needed within the standard for the source input character set. Further, on the execution side, it should not be assumed that code points are encoded. A term that is more abstract than code point is needed here.
- Hubert agreed that numeric code point values are not needed, but noted that abstract character isn't necessarily the right term either.
- Corentin stated that code point could imply a numeric value, but that the standard need not discuss it.
- Tom replied that, in ISO/IEC 10646 and Unicode, code point is primarily defined as a numeric value.
- Hubert observed that, if the internal character set is specified to be Unicode, then there is no requirement to define what a "character" is, but use of a term like "extended character" will require avoiding discussion of details since they would be implementation-defined.
- Jens observed that implementations could use code point values above 0x10FFFF for extended characters.
- Jens added that there is benefit to being aligned with C if we were to adopt the C99 model.

- Jens opined that there is no benefit in requiring the internal character set to be isomorphic to Unicode.
- PBrett stated that the alternative to an internal character set is Unicode and expressed a preference that, if the internal character set is effectively Unicode, that it just be made Unicode.
- Hubert responded that the goal was to avoid formation of UCNs in translation phase 1 and that doing so results in having to handle extended characters. That implies that the internal character set must map Unicode or Unicode plus additional implementation-defined characters.
- **Poll: We generally believe that the internal character set should be Unicode based, but that implementations can support non-Unicode characters.**
 - **Attendees: 10**
 - **S F F N A S A**
2 5 1 2 0
 - A: If non-Unicode characters are allowed, then we are not encouraging migration to Unicode and portability.
 - A: People with more expertise than us have been defining characters for all humanity and this poll states that isn't sufficient.
- Hubert responded to the against positions stating that the intent is not to change the behavior of current programs and the against positions are therefore not consistent with the intent.
- **Poll: We want to transition away from forming UCNs in phase 1 in favor of plumbing extended characters (perhaps as specified by C99)**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
- Tom asked if anyone would be willing to volunteer to summarize the mechanism used in C and post it to the mailing list.
- Corentin volunteered.
- Tom confirmed that the next meeting will be on July 8th.
- PBindels reminded the group that EWG is scheduled to review [P1949R4](#) the following day (Thursday, 2020-06-18).

July 8th, 2020

Draft agenda:

- Continue discussion of terminology updates to strive for in C++23
 - Determine suitability of ISO/IEC 10646 terms for use in the C++ standard.
 - Character
 - Repertoire
 - Code point
 - Coded character
 - Coded character set
 - Code unit
 - Code unit sequence
 - Encoding form
 - Encoding scheme
 - UCS codespace
 - UCS scalar value
 - Well-formed code unit sequence
 - Minimal well-formed code unit sequence
 - Ill-formed code unit sequence

- Ill-formed code unit sequence subset
- Identify possible terms to add to [\[intro.defs\]](#).

Attendees:

- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Walter Brown
- Zach Laine

Meeting summary:

- Discussion of the suitability of [ISO/IEC 10646:2017](#) terms for use in the C++ standard
 - Tom introduced the topic:
 - The intent is to focus on terminology, determine what terms from ISO/IEC 10646 are usable in the C++ standard and for what purposes, and what new terms will be needed.
 - Zach advised against introducing new terms or redefining existing terms with different meanings.
 - Hubert agreed that if we try inventing terms, then we risk causing some of the same problems that the Unicode consortium did by making terms overly specific; we want generic terms.
 - PBrett also agreed and noted that we don't want to create an N+1 specification.
 - Jens stated that there may not be much reason for concern; the proposed wording for [P2029](#) illustrates that we can avoid the need for some terms. For example, we may be able to get rid of execution character set completely by only discussing an execution encoding rather than a character set.
 - PBrett asked Jens to confirm that only character encodings can be observed, not character sets.
 - Jens replied, yes.
 - The group proceeded to discuss terms from ISO/IEC 10646.
 - **character:**

member of a set of elements used for the organization, control, or representation of textual data

Note 1 - A graphic symbol can be represented by a sequence of one or several coded characters.

- Jens commented that he used to believe that ISO/IEC 10646 matched the Unicode standard, but the ISO/IEC 10646 terms differ from Unicode.
- Tom acknowledged and relayed his understanding that we are required to refer to ISO standards when they exist, so we need to first consider the terms from ISO/IEC 10646.
- Jens confirmed that understanding.
- Hubert asked where we envision using the "character" term from ISO/IEC 10646 in the standard.
- Jens replied that we need a term for the members of the basic source character set and for the input source.
- Tom added that we may need the term for the entity that is designated by a simple escape sequence.

- Jens responded that, since simple escape sequences designate an execution time value, that entity can be a code unit sequence.
- Hubert noted that all of the characters designated by simple escape sequences only require a single code unit, not even a code unit sequence.
- Hubert noted that the designated code units do have associated semantics however; like BEL for example.
- Jens replied that semantics can be established by referring to the character name or to a Unicode code point.
- Hubert expressed support for the generality of that approach since it is required that the mapping to execution encoding can't fail.
- PBrett asked if there is a need for the concept of a character for locale purposes.
- Jens replied that there may be, but that we should just focus on core language for now and locale is all run-time.
- Mark observed that `std::basic_string` defines character in its own way.
- Zach asked if "character" will be needed in order to define other terms and noted that any dependencies will need to be resolved in the standard.
- Tom replied that any dependent terms are already available via the existing reference to ISO/IEC 10646.
- Jens stated that the list of terms in the telecon agenda are ones that we should try not to add to [\[intro.defs\]](#) as the existing terms that are there are not particularly useful.
- Walter agreed and noted that the existing terms are somewhat enemic.
- Hubert stated that not putting terms in [\[intro.defs\]](#) is concerning unless wording is specific about where used terms come from.
- Tom asked if there is a way that we can be explicit about where terms come from.
- Hubert responded that we haven't done that previously.
- Walter suggested that can be investigated offline.
- **repertoire:**

specified set of characters that are represented in a coded character set

- Tom observed that the definition has an explicit dependency on "coded character set".
- Jens stated that the dependency makes that term unusable for our purposes since it isn't sufficiently abstract.
- Hubert agreed.
- Jens stated that a term is needed for the abstract entities that form the source input.
- Tom summarized the observations by stating that this term and its definition can't be used, but we recognize a need for a term that doesn't have a dependency on "coded character set".
- Steve noted that we can't adopt terms from the C standard because they have a different character model; we use the same terms to mean different things. The [C99 rationale document](#) exposed this.
- Jens agreed and commented that the current C++ model needs to change towards something more like the C model, but the C model wording predates Unicode and doesn't use modern terminology.
- **code point:**

value in the UCS codespace

- Tom decreed that the definition is terrible since it requires "UCS codespace".
- Jens read the definition of "UCS codespace".
- Jens noted that "UCS codespace" includes surrogate code points.

- Zach stated that surrogate inclusion is intentional, but people often use code point where scalar value is intended; we'll need more precision in wording.
- Tom asked if an analogue of code point for non-Unicode encodings is needed.
- Jens replied no, only code units are needed; even for character literals.
- Hubert expressed some uncertainty and that something like code point may be needed for *universal-character-names* (UCNs).
- Jens summarized Hubert's concern and stated that UCNs are a sequence of characters that designate a scalar value and that we need to be able to state that the universal character set maps to Unicode code points.
- Steve mentioned short-identifier syntax, `u+xxxx`, and noted that, in a UCN, the `xxxx` is the short-identifier.
- Jens replied that short-identifier syntax is problematic because of restrictions on leading 0s; Unicode only allows leading 0s to pad to a maximum length of 6 digits, but UCNs require a length of exactly 4 or 8 digits.
- Jens noted that the "code point" term and its definition can be used, but only in a Unicode context.
- **coded character:**

association between a character and a code point

- Tom noted the term is Unicode specific due to the use of "code point" in the definition.
- Jens agreed and noted the same condition for "coded character set", but emphasized that neither appears to be needed for the C++ standard since only code units and code unit sequences are observable.
- PBrett agreed.
- **code unit:**

minimal bit combination that can represent a unit of encoded text for processing or interchange

Note 1 - Examples of code units are octets (8-bit code units) used in the UTF-8 encoding form, 16-bit code units in the UTF-16 encoding form, and 32-bit code units in the UTF-32 encoding form.

- Tom excitedly noted that this definition is not Unicode specific.
- Hubert agreed and added that it can be used to describe the contents of strings, including wide strings.
- Tom asked if there are any places other than strings where code unit sequence would be relevant.
- Jens replied that there are definitely use cases in the library.
- PBrett asked about the requirement that the values of the characters "0" through "9" in the execution character set be contiguous.
- Hubert replied that that requirement can be defined in terms of code units.
- Jens commented that in other wording he is involved with, that just integer value suffices since `char`, `wchar_t`, etc... are all integer types.
- PBrett recounted claims from others in outside conversations that it may have been a mistake to define the character types as integer types and suggested that, in a rewrite, it may be beneficial to avoid that.
- Jens agreed, but noted that for backward compatibility, a rewrite would have to allow conversions.

- PBrett suggested that it is useful to be able to distinguish between a code unit and an integer value.
- Hubert noted that we would still need to discuss integer values because `char` and `wchar_t` have implementation-defined signedness.
- Jens agreed and stated that other such restrictions exist.
- Zach stated that, in the library wording, having definitions is very useful since the library environment tends to be less abstract.
- **code unit sequence:**

element of interchanged information that is specified to consist of a sequence of code units, in accordance with one or more identified standards for coded character sets

Note 1 - Such sequence can contain code units associated with any type of code points.

Note 2 - Since its second edition: ISO/IEC 10646:2011, this International Standard does not use implementation levels. Its definition of code unit sequence corresponds to the former unrestricted implementation level 3. Other definitions of code unit sequence, previously known as level 1 and 2, are deprecated. To maintain compatibility with these previous editions, in the context of identification of coded representation in International Standards such as ISO/IEC 8824 and ISO/IEC 8825, the concept of implementation level can still be referenced as 'Implementation level 3'. See Annex N

- Tom observed that this definition appears to require an association with a standard.
- Zach expressed a lack of concern; EBCDIC can be considered a "standard" for this purpose.
- PBrett agreed and stated the same is true for WTF-8.
- Hubert noted that ISO/IEC 10646 may not have the ability to declare something as "implementation-defined", hence a deference to a standard.
- Tom asked for confirmation that this definition is ok for our purposes.
- Jens agreed that it is.
- Walter expressed frustration with the discussed terms and definitions being so circular and asked where terms and definitions that don't depend on prior knowledge might be found.
- Jens responded that, in a standard, definitions should generally be presented at the beginning of the standard and explained by later prose.
- Hubert noted that the quality of these definitions is such that expectations of helpful prose later in the document may lead to disappointment.
- Zach commented that people end up developing a working knowledge of these terms and processes, but the ability to define them well remains elusive.
- Tom lamented a better source of terminology and noted that the reason we are discussing these is exactly because a good agreed upon source of terms is not readily available.
- Jens asserted that this is good motivation for reducing usage to as few terms as possible.
- PBrett agreed and added that "character" should be especially avoided because it probably has the most fuzzy connotations.
- **encoding form:**

form that determines how each UCS code point for a UCS character is to be expressed as one or more code units used by the encoding form

Note 1 - This International Standard specifies UTF-8, UTF-16, and UTF-32.

encoding scheme:

scheme that specifies the serialization of the code units from the encoding form into octets

Note 1 - Some of the UCS encoding schemes have the same labels as the UCS encoding form. However, they are used in different contexts. UCS encoding forms refer to in-memory and application interface representation of textual data. UCS encoding schemes refer to octet-serialized textual data.

- Jens stated that encoding scheme is relevant for encoding of octets in big-endian vs little-endian order, and that encoding form is for code units.
 - Jens added that encoding scheme is unnecessary for our purposes since endian issues are not specified.
 - Jens further added that encoding form is unnecessary since encodings such as UTF-8, UTF-16, and UTF-32 can be referred to by name.
 - Mark asked if encoding form might be needed for literals.
 - Jens replied that implementation-defined encoding or mention of a specific encoding name suffices.
 - Tom noted that specific encoding names will be needed for the implementation-defined encodings for Corentin's [P1885](#) proposal to expose the encoding used for literals and by the locale, but agreed not for core language.
 - Tom summarized; consensus seems to be that we don't need encoding form, encoding scheme, or analogues for non-Unicode encodings.
 - Zach agreed and noted that "encoding" can be used without intruding on "encoding form".
- **UCS codespace:**

codespace consisting of the integers from 0 to 10FFFF (hexadecimal) available for assigning the repertoire of the UCS characters.

UCS scalar value:

any UCS code point except high-surrogate and low-surrogate code points

- Tom stated that both "UCS codespace" and "UCS scalar value" are available for use in Unicode contexts.
 - Jens agreed.
 - Mark noted that these terms start with "UCS" and that, colloquially, that prefix isn't generally used, but that the standard should specifically use the UCS prefixed terms.
 - Jens agreed and added these terms don't appear frequently enough to warrant a shorter term.
 - Jens added that "scalar value" by itself is not specific enough anyway.
- **well-formed code unit sequence:**

UCS code unit sequence that purports to be in a UCS encoding form which conforms to the specification of that encoding form and contains no ill-formed code unit sequence subset

minimal well-formed code unit sequence:

well-formed code unit sequence that maps to a single UCS scalar value

- Jens stated that neither of the "well-formed" terms are interesting for core language.

- Tom countered that these could potentially be useful for a fully specified translation phase 1 for Unicode encoded source files.
- PBrett stated that, absent implementation defects, it is not possible for literals to not be well-formed.
- Zach expressed uncertainty.
- Hubert noted that, for source input, all that exists are characters and UCNs, so yes, well-formedness is assured.
- Steve agreed and added that we've previously agreed that ill-formed code unit sequences in literals are possible due to numeric escape sequences, but that the input to the literal encoding is always well-formed.
- Mark expressed surprise that these terms are not needed in the code language.
- Tom replied that library will eventually need these terms or analogous ones.
- Zach agreed that we should revisit these terms for library.
- **ill-formed code unit sequence:**

UCS code unit sequence that purports to be in a UCS encoding form which does not conform to the specification of that encoding form
 EXAMPLE - An unpaired surrogate code unit is an ill-formed code unit sequence.

ill-formed code unit sequence subset:

non-empty subset of a code unit sequence X which does not contain any code unit which also belong to any minimal well-formed code unit sequence subset of X

Note 1 - An ill-formed code unit sequence subset cannot overlap with a minimal well-formed code unit sequence.

- Tom stated that the situation is the same for the "ill-formed" cases as for the "well-formed" ones; they can be used in library, but are not needed for core language.
- Tom stated that this meeting concludes our discussion of terminology for now and that a paper will be needed to make more progress.
- Tom stated that the next meeting will be on July 22nd and will discuss [P2178](#).

July 22nd, 2020

Draft agenda:

- [P2139R2: Reviewing Deprecated Facilities of C++20 for C++23](#)
 - Provide recommendations for D.20-D.23.
- [P2201R0: Mixed string literal concatenation](#)
 - Validate consensus to encourage that this paper be forwarded directly to core.
- [P2178R1: Misc lexing and string handling improvements](#)
 - Begin discussions on the various proposals.
 - Possibly begin taking direction polls.

Attendees:

- Alisdair Meredith
- Corentin Jabot

- Jens Maurer
- Martinho Fernandes
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Tom provided some administrative updates:
 - Tom now has a Zoom account setup courtesy of the ISO.
 - SG16 telecons will switch to Zoom starting with the next telecon on August 12th.
- [P2139R2: Reviewing Deprecated Facilities of C++20 for C++23](#):
 - Alisdair provided an introduction.
 - LEWG has already discussed the proposed changes.
 - In general, LEWG is in favor of removal of the deprecated features since implementors can continue to provide them due to the zombie clause ([\[zombie.names\]](#)).
 - D.20: Deprecated Standard code conversion facets [`depr.locale.stdcvt`]
 - *[Editor's note: This concerns the `codecvt` facets that convert between UCS-2, UTF-8, UTF-16, and UTF-32; `codecvt_utf8`, `codecvt_utf8_utf16`, and `codecvt_utf16`.]*
 - Alisdair stated that these interfaces are all underspecified; the wording was based on Dinkumware's documentation.
 - Alisdair indicated that the reference to UCS-2 in the wording for these facets is all that is preventing us from removing the normative reference to ISO/IEC 10646:1993. UCS-2 has been deprecated for 20 years and the ISO no longer provides a standard with a definition for it.
 - *[Editor's note: According to [chapter 2 of Unicode 13](#), UCS-2 was removed from ISO/IEC 10646 in ISO/IEC 10646:2011.]*
 - Jens agreed that uses of the UCS-2 term and normative reference to an outdated standard should be removed.
 - PBrett directed the group to [P0618](#), the paper that deprecated these features and noted that there were recent complaints by a few committee members about deprecating these features. JeanHeyd is now working on a replacement.
 - *[Editor's note: The paper trail for P0618 is a little difficult to follow. The paper was written to address C++17 NB comment GB 57. LEWG consensus for resolving GB 57 by deprecating the `<codecvt>` header was by unanimous consent at the Issaquah 2016 meeting.]*
 - Zach responded that the concerns about deprecation may be abstract; that only features that are actively harmful should be removed. Disliking a feature is not sufficient grounds for deprecation.
 - PBrett noted that the referenced committee members are under the impression that the `codecvt` facets work; at least for basic uses.
 - Alisdair stated that their concern was deprecation without a replacement.
 - Tom noted that the discussion around those complaints was confusing. Some of the code posted that worked on one platform but not another was using `std::codecvt` specializations that have never been guaranteed to exist by the standard. The code in question wasn't using the deprecated facets at all.
 - Steve stated that these facets are an attractive nuisance; we have evidence that people have a hard time using them and that trying to use them for UTF-16 often leads to bad bugs.

- Jens stated that there are differences of opinion regarding what deprecate means. For example, comments have been made that deprecating `std::regex` is intended to invite alternate proposals. But deprecation may lead to the addition of `[[deprecated]]` attributes which may result in warnings which may be elevated to errors which may cause problems for programmers.
- Jens added that we should have a migration path, but we don't have replacements yet.
- Jens asked if we can salvage these interfaces, at least the parts that convert between UTF-8 and UTF-16.
- Alisdair responded that the interfaces don't consistently convert to UCS-2 vs UTF-16.
- Jens asked if we can just remove the functionality that relates to UCS-2.
- Corentin commented that the scope of the paper is deprecation or removal and stated that we should not consider other options.
- PBrett agreed with Corentin.
- Alisdair replied that the intent of the paper is to find good direction and that he is happy to consider other options.
- Tom suggested that a poll on other approaches might be useful.
- PBrett stated that his primary concern with `codecvt` is that error handling is poor.
- Zach stated that he has only used these facets once and asked if they produce replacement characters for ill-formed code unit sequences.
- Alisdair responded that we don't know because the feature is so underspecified.
- Zach stated that removal is preferred if these don't conform to expected Unicode behavior and conformance requirements.
- Alisdair asked if anyone other than Jens is in favor of trying to remove just the UCS-2 support.
- Tom indicated weak support.
- Jens expressed concern about removal without replacement and pondered whether these should have been deprecated at all.
- PBrett indicated that he was originally surprised by the deprecation, but that the rationale for doing so made sense.
- PBrett added that people will continue to try to use these features if they are retained.
- Tom asked what the real life impact is of removal vs deprecation.
- Alisdair responded that it depends on what implementors choose to do. Some may hide the interfaces behind macros while others leave them in place. Similar cases in the past have lead to portability issues.
- Zach noted that the interfaces might be annotated as removed at cppreference.com.
- PBrett noted some indications that some systems are built with the deprecated features removed.
- Tom responded that those may be misunderstandings; `libstdc++` limits the available `std::codecvt` facets to specializations specified by the standard such that use of unknown specializations leads to linker errors.
- Victor stated that the choice should be pretty clear here; these features are poorly designed and should be removed.
- Corentin noted that LEWG has already indicated desire to remove and is just looking for confirmation.
- **Poll: The deprecated Standard code conversion facets specified in D.20 `[depr.locale.stdcv]` should be removed.**
 - Attendees: 9
 - SF F N A SA
 - 3 3 1 2 0

- Consensus is for removal.
- D.21: Deprecated convenience conversions [depr.conversions]
 - *[Editor's note: This concerns the `wstring_convert` and `wbuffer_convert` class templates.]*
 - Alisdair explained that these interfaces were deprecated at the same time as the interfaces in D.20, that the current wording has a dependency on those interfaces, that the wording could be updated to avoid that dependency (as demonstrated in the paper in the proposed wording for D.20), and that the urgency to remove these is therefore not as strong as for D.20.
 - PBrett observed that the motivation for deprecating these is not explained in the paper that proposed their deprecation, [P0618](#).
 - Alisdair responded that he does not recall there being strong motivation for deprecation other than their association with the `codecvt_utf8` and `codecvt_utf8_utf16` facets.
 - PBrett expressed some concern about removal given that they can still be used with the non-deprecated `codecvt` facets.
 - Tom noted that there are some locale restrictions; these interfaces can't use a locale managed `codecvt` facet.
 - Jens responded that it looks like it only requires no side effects that impact locale.
 - Corentin agreed with Peter's concerns; these interfaces aren't particularly harmful or confusing.
 - Alisdair asked if un-deprecating these should we considered.
 - Jens replied that a suitable replacement that handles errors properly is likely to have a different interface, so un-deprecating these is probably not the right choice without other motivation.
 - Zach noted that these interfaces don't appear to be an active problem; no one uses them accidentally.
 - Steve asked if the question to SG16 should be whether we object to removal.
 - Alisdair responded that he heard more informed discussion in the last five minutes than he had in LEWG.
 - Jens opined that removal is under-motivated.
 - Alisdair asked if there would be more support for removal if a replacement was available.
 - A chorus of affirmations was heard.
 - Alisdair responded favorably and noted that features should not be left in annex D perpetually.
 - **Poll: The deprecated convenience conversions specified in D.21 [depr.conversions] should be removed.**
 - **Attendees: 9**
 - **SF F N A SA**
 - **0 1 6 2 0**
 - Consensus is for no change to status quo.
 - **Poll: Does SG16 object to removal of the deprecated convenience conversions specified in D.21 [depr.conversions]?**
 - **Attendees: 9**
 - **Yes No**
 - **1 8**
 - Consensus is no objection.
- D.22: Deprecated locale category facets [depr.locale.category]
 - *[Editor's note: This concerns the `char-based UTF-8 codecvt` and `codecvt_byname specializations`.]*
 - Alisdair mentioned that this deprecation came from SG16.

- Tom explained that these facets were deprecated with the introduction of `char8_t`; the deprecated specializations squat on the interfaces that would be desired for conversion between the locale dependent narrow encoding and either UTF-16 or UTF-32.
- Tom stated that we don't know what will happen with `char8_t`, particularly in the Linux community where the narrow locale is dependably UTF-8; projects that build with `char8_t` support disabled may benefit from preserving these.
- Jens noted that these specializations were just deprecated in C++20.
- Tom stated that retaining these may be useful for code that needs to be compatible across C++17 and C++23, perhaps in projects that introduce a typedef as conditionally `char` or `char8_t`.
- Alisdair observed that zombification may not be a good answer in that case.
- PBrett asked how likely it is that we would want to re-use these specializations.
- Tom responded that it is not very likely; we want to move away from `std::codecvt`.
- Zach agreed.
- Steve predicted that the repurposed specializations would probably only be used with the `wstring_convert` and `wbuffer_convert` interfaces which may be removed soon.
- Alisdair observed that these specializations don't become zombies because they are just specializations, not names.
- PBrett asked what LEWG's inclination was.
- Alisdair responded that it was to remove and depend on the zombie clause.
- **Poll: The deprecated locale category facets in D.22 [`depr.locale.category`] should be removed.**
 - **Attendees: 9**
 - **S F F N A S A**
1 2 2 1 2
 - Consensus is for no change to status quo.
 - SF: I'm not empathetic towards the argument that people may not use `char8_t` on Linux, nor do I find the typedef compatibility approach compelling.
 - SA: I'm concerned about ease of writing code that is compatible across C++17 and C++23.
- **Poll: Does SG16 object to removal of the deprecated locale category facets in D.22 [`depr.locale.category`]?**
 - **Attendees: 9**
 - **Yes No**
1 8
 - Consensus is no objection.
- D.23: Deprecated filesystem path factory functions [`depr.fs.path.factory`]
 - *[Editor's note: This concerns `std::filesystem::u8path`.]*
 - Alisdair explained that `u8path` only existed because `char8_t` wasn't available to differentiate constructor declarations for narrow encoding vs UTF-8; the `char8_t` constructor is now available.
 - Alisdair added that LEWG's inclination is to remove the function and rely on the zombie clause for backward compatibility.
 - Jens asked what the LEWG quorum was for the discussion.
 - Alisdair responded that there were about 30 attendees with good breadth of experience but not necessarily depth.
 - Corentin opined that this removal is not really an SG16 matter and is more traditional LEWG territory.
 - PBrett agreed that this isn't really an SG16 matter.

- Jens noted that a replacement is available but opined that removal is premature since this was just deprecated in C++20.
- Alisdair noted that the function was just added in C++17, so hasn't been around much.
- Tom commented that the same concerns about C++17 and C++23 compatibility discussed for the deprecated `codecvt` specializations applies here.
- **Poll: Does SG16 object to removal of the deprecated filesystem path factory functions in D.23 [depr.fs.path.factory]?**
 - **Attendees: 9**
 - **Yes No**
 - 0 9**
 - Consensus is no objection.
- [P2201R0: Mixed string literal concatenation](#):
 - Jens introduced the paper.
 - This makes mixed encoding string literal concatenation ill-formed.
 - The only compiler known to implement this conditionally-supported implementation-defined behavior is the [SDCC](#) C compiler. No C++ compilers are known to support it.
 - Tom stated that the intent is, assuming consensus, to forward this paper directly to the CWG assuming agreement by the EWG chair.
 - **Poll: Direct Tom to recommend to the EWG chair that P2201R0 be forwarded directly to the CWG.**
 - **Attendees: 9**
 - **S F F N A S A**
 - 8 1 0 0 0**
 - Consensus is to forward to the CWG.
- Tom stated that the next telecon will be held August 12th and will discuss [P2178R1](#).

August 12th, 2020

Draft agenda:

- [P2178R1: Misc lexing and string handling improvements](#)
 - Begin discussions on the various proposals.
 - Possibly begin taking direction polls.

Attendees:

- Corentin Jabot
- Hubert Tong
- Mark Zeren
- Martinho Fernandes
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Walter Brown
- Zach Laine

Meeting summary:

- Tom provided an administrative update:
 - The EWG chair declined forwarding [P2201R0: Mixed string literal concatenation](#) directly to the CWG in order to avoid any possible appearance of unilateral decision making. The paper will be reviewed during the EWG telecon on August 19th.
- [P2178R1: Misc lexing and string handling improvements](#)
 - Tom stated that the proposals will not be discussed in the order presented in the paper as proposals 1 and 9 are complicated and/or contentious. The goal is to provide feedback quickly on the proposals that are unlikely to be contentious so that progress can be made on those without being held up by the others.
 - Hubert asked if support for proposal 1, mandated support for UTF-8 as a source file encoding, could be handled by EWG without SG16 holding it up.
 - Tom responded that there are technical details and possible points of contention that should be worked out in SG16 first.
 - Corentin provided an overview of the paper.
 - The paper presents a number of proposals intended to address issues identified with current lexing behavior and wording.
 - As prior discussion has revealed, lack of consistent terminology leads to confusion; we need to ensure the
 - underlying model is commonly understood.
 - Many of the issues address concerns that are especially significant for Unicode support.
 - The proposals are bundled into a single paper due to interconnected concerns.
 - Proposal 2: What is a whitespace or a new-line?
 - Corentin stated that this is intended to align with Unicode specifications for what constitutes whitespace.
 - Corentin added that the motivation is to move away from implementation-defined behavior in phase 1.
 - PBrett asked if this proposal is separable from the others; the introduction argues for considering all of these proposals collectively.
 - Corentin replied that he would like to have just one paper for wording.
 - PBrett acknowledged that goal but repeated the question as to whether separation is possible.
 - Corentin replied that separation is possible, but that the individual proposals have less value, and therefore little urgency to address, when considered individually rather than collectively.
 - Corentin asked what the semantics should be for a raw string literal and whether the exact line termination sequence should be preserved.
 - Tom replied that there is a core issue for that.
 - Corentin acknowledged and noted that it is mentioned in the paper ([CWG #1655](#)).
 - Proposal 3: Preserve Normalization forms
 - Corentin stated that the intent is to standardize existing practice and to persist source information through translation phases 1 and 5.
 - Tom asked if this proposal is dependent on proposal 1 and then answered his own question in the negative.
 - Zach noted that there is a dependence on knowing what the source encoding is.
 - Corentin replied that the compiler knows what encoding is being used.
 - Zach acknowledged, but noted that the compiler has to be informed, so stating that it knows the encoding is vacuous.
 - Corentin stated that the intent is that, if the source is UTF-8, that code points are preserved.

- Zach responded that we previously determined that we can't reliably determine when the encoding being used does not match; there needs to be a portable way to indicate the source encoding.
- [*Editor's note: that determination was made during discussions of [P1879](#).]*
- Hubert stated that this just requires that the implementation specifies the encoding that is being used for the source input.
- Tom asked if normalization form is the right concern; preservation of code points would address the more general concern.
- PBrett noted that this proposal is separable from proposal 1 because the implementation knows the encoding that is being used.
- PBrett added that this proposal is applicable for all encodings since non-basic source characters are mapped to *universal-character-names* (UCNs).
- Tom requested that the paper address the case where the execution encoding supports é as a combined character (e.g., U+00E9 {LATIN SMALL LETTER E WITH ACUTE}), but not as separate characters (e.g., U+0065 {LATIN SMALL LETTER E} followed by U+0301 {COMBINING ACUTE ACCENT}).
- Zach opined that this proposal should still be coupled with proposal 1.
- Tom replied that Peter's explanation seems sufficient to describe how this would work in an encoding agnostic way.
- Zach stated that requires knowing what the source encoding is.
- Hubert noted that discussion of codepoint-by-codepoint translation is challenging without more structure around translation phase 1.
- Proposal 4: Making trailing whitespaces non-significant
 - Corentin stated that this is a lexing concern, but not really a Unicode or text concern.
 - Corentin explained that gcc defends its removal of trailing white space as part of its translation phase 1 semantics.
 - Corentin noted that Microsoft Visual C++ behavior diverges from gcc and clang.
 - Corentin added that editors may implicitly remove trailing whitespace; semantically meaningful trailing whitespace is therefore fragile.
 - Corentin summarized; the proposal is to align the standard with the behavior exhibited by gcc and Clang and to ignore trailing white space for the purposes of determining line continuation.
 - Hubert observed that proposal 2 seeks to do the opposite of the intent for this proposal by potentially preserving the form of line endings, at least in raw string literals.
 - Hubert added that the usual way this elision of trailing whitespace is handled is by claiming that the preceding white space is considered part of the line termination.
 - Tom asked if there had been any comments from Microsoft implementors given that a change here would presumably require a change to their implementation.
 - Corentin responded that he had reached out, but didn't hear back.
- Proposal 5: Restricting multi-character literals to members of the Basic Latin Block
 - Corentin noted that multi-character literals are used and this is not a proposal to removing them.
 - Corentin explained that multicharacter literals that present as a single character are confusing, for example 'é' written with a combining character.
 - Corentin added that implementations diverge in their handling of them.
 - Corentin stated that the proposal intent is to make such confusing cases ill-formed.
 - PBrett expressed support for this direction.
 - Tom asked why the restriction is to one code point.
 - Corentin replied that the intent is that each character in the literal be limited to, effectively, ASCII.
 - Mark asked why the 4th example is not ok given that the 2nd and 3rd examples are.

- [Editor's note: the 2nd example is 'abc', the 3rd is '\u0080', and the 4th is '\u0080\u0080'.]
- Corentin responded that the 3rd example is not a multicharacter literal, but the 4th is. The 4th is excluded because it contains *c-chars* that identify characters outside the Unicode basic Latin block.
- PBrett opined that cases like the 2nd example are used, but that cases like the 4th are not and have no known use cases.
- Hubert observed that the examples are incomplete without octal and hex escapes.
- Tom expressed difficulty trying to understand how to separate between the basic source character and UCN examples.
- Hubert suggested that some presentation improvements might make the examples easier to understand.
- Hubert expressed support for allowing octal and hex escapes within multicharacter literals.
- Tom, still trying to comprehend the examples, expressed a belief that he was reading far too much into the use of UCNs in the example.
- PBrett stated that the use of UCNs is intended to make it more clear exactly which character is designated.
- PBrett suggested either adding or changing the examples for the next revision.
- Mark observed that broken UTF-8 is allowed in string literals, but that this is kind of different.
- Tom disagreed and noted that numeric escapes would not get transcoded, but would still contribute a value to the appropriate "slot" in the `int` value.
- Tom asked if the size of `int` is relevant. For example, if `sizeof(int)` was 2, would the number of *c-chars* allowed in the multi-character literal be limited to 2?
- Corentin responded that no, that would still be implementation-defined; the intent is just to address the visual confusion.
- Mark noted that this is technically a breaking change, but that numeric escapes can be used as a work around.
- Corentin responded affirmatively, but noted the concern is mostly theoretical; he hasn't been able to find any examples that would be disallowed by these changes.
- Hubert noted that swapping in a numeric escape could change behavior and therefore should not be suggested as a compiler fixit hint.
- Proposal 6: Making wide characters literals containing multiple or unrepresentable *c-char* ill-formed
 - Corentin explained that wide multicharacter and non-encodable character literals are inherited from C.
 - Corentin noted that there is implementation divergence; some compilers produce warnings and some do not.
 - Mark observed that the paper does not include data from code searches.
 - Corentin responded with uncertainty whether he had conducted code searches for this proposal.
 - Tom recalled possibly seeing these used with Visual C++ and `TCHAR`.
 - Corentin stated that he can't say with certainty that these are not used.
 - Hubert noted that Corentin's research indicates these don't behave like ordinary multi-character literals.
 - PBrett stated that the different behavior contradicts Tom's recollections.
 - Tom suggested that his recollection is likely incorrect.
 - Tom stated that the motivation for this proposal seems somewhat different than for the previous proposal; this proposal isn't just about avoiding visual confusion.
 - PBrett replied that it is similar; the motivation for the prior case applies here, but is compounded by the fact that all but one of the *c-chars* in the literal are ignored in this case.
 - Tom acknowledged but noted that is similar to the previous case too where excess *c-chars* are ignored.

- Proposal 7: Making conversion of character and string literals to execution and wide execution encoding ill-formed for unrepresentable c-char
 - Corentin explained that Clang rejects such conversions and Visual C++ substitutes a '?'. According to Billy O'Neal, the replacement with a question mark is due to the default behavior of the conversion functions used.
 - Tom stated that the paper should be updated to add a reference to [P1854](#).
 - Tom continued; in Belfast, an example was discussed of checking if a character in a literal is converted to a specific value in order to infer the execution encoding.
 - Tom provided an example:

```
"\u1234" == 0x1234
```

- Hubert suggested an alternative syntax for fun:

```
__try__("\u1234") == 0x1234 // :)
```

- Corentin stated that this seems like a different issue.
- Tom agreed, but noted that making non-encodable characters ill-formed means such checks can no longer be performed. The intent is to allow code to use some characters if available and to fallback otherwise.

```
if ('\u1234' == 0x73) {
    return '\u1234';
} else {
    return 'X';
}
```

- Pbrett noted that this presents a trade off for a small number of people who care about clever tricks like that vs the many more programmers that might experience surprising behavior.
- Zach observed that the code presented presumably doesn't work for gcc and clang.
- Tom replied that gcc will accept it depending on whether `-finput-charset` and/or `-fexec-charset` are specified; if gcc has to get iconv involved, then an error may be reported.
- Tom added that the trade off is the important concern here, not the use case; the use case can be addressed in other ways.
- [P1949: C++ Identifier Syntax using Unicode Standard Annex 31](#):
 - Tom asked Steve if he had any updates to share since the EWG review.
 - Steve replied that he was without power for a while but that he would try to get an update into the August mailing.
- Tom stated that the next meeting will be August 26th and that we'll continue discussing [P2178R1](#) starting with proposal 8.
- Tom reminded the group that Jen's paper, [P2201R0: Mixed string literal concatenation](#), will be presented to EWG on August 19th.

August 26th, 2020

Draft agenda:

- [P2178R1: Misc lexing and string handling improvements](#)
 - Continue discussions on the various proposals in the order 8, 10-12, 1 (discussion of proposal 9 will be deferred due to the arrival of P2194R0).

- Begin taking direction polls.

Attendees:

- Corentin Jabot
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2178R1: Misc lexing and string handling improvements](#)
 - Proposal 8: Enforcing the formation of universal escape sequences in phase 2 and 4
 - Corentin stated that these cases of undefined behavior are surprising; defining behavior would not appear to present a problem to implementations.
 - Corentin added that gcc, Clang, Visual C++, and the EDG based Intel C++ compiler all exhibit the same behavior.
 - Corentin mentioned that SG12 should be consulted.
 - Corentin asserted that, if defining portable behavior presents a challenge, then the standard should specify that behavior is implementation-defined.
 - Hubert stated that the undefined behavior is present to accommodate various preprocessor models; early models recognized *universal-character-names* (UCNs) in translation phase 1 and did not check for them again after translation phase 2 (logical line formation) or translation phase 4 (macro expansion and token pasting); the differences are observable.
 - Hubert noted that there are many C implementations, so WG14 may not be interested in defining this behavior.
 - Jens stated that preprocessor undefined behavior falls under SG12, but he is unaware of any activity addressing this specific issue.
 - Jens asserted that both SG12 and WG14 should be informed of any efforts here.
 - Jens noted that defining behavior just for C++ does not impact compatibility with C.
 - Tom stated that this is not an SG16 concern.
 - Corentin agreed.
 - Proposal 10: Make L in `_Pragma` ill-formed
 - Corentin explained that `_Pragma` expressions written with a wide string literal are well-formed in both C and C++, but are semantically identical to an expression written with an ordinary string literal.
 - Corentin added that C also permits the string literal to be written with `u8`, `u`, and `U` encoding prefixes as well; C++ only allows `L`.
 - Corentin stated that the intent is to make the presence of an encoding prefix ill-formed since it serves no semantic purpose.
 - PBrett agreed with the direction and stated that an encoding prefix being present only leads to confusion.

- Tom asked if it matters that `_Pragma` is processed in translation phase 4, but that tokenization is performed in translation phase 3.
- Hubert responded that it would for raw string literals.
- PBrett asked if raw string literals are allowed.
- Hubert expressed uncertainty.
- Corentin stated that when WG14 adopted support for the `u` and `U` encoding prefixes, they systematically added them everywhere that the `L` encoding was allowed; C++ did not do likewise.
- Jens stated that failure to add the additional encoding prefixes in C++ was an oversight.
- Jens noted that `_Pragma` accepts a *string-literal* and that includes *raw-string*.
- Jens asserted that this is not SG12 territory, but is liaison territory with WG14.
- PBrett noted that this is technically evolutionary.
- Corentin stated that this is not really an SG16 concern.
- Tom agreed; there is no actual encoding here.
- PBrett asked for confirmation that these strings are interpreted directly by the compiler.
- Mark asked if the compiler observes the source encoded string.
- Corentin replied that the compiler observes the string in the internal encoding.
- Tom agreed and noted that the observation occurs after translation phase 1 (conversion to internal encoding) and before translation phase 5 (conversion to execution character set).
- Jens opined that the use of *string-literal* is a hack to align behavior with `#pragma`.
- JeanHeyd asked for confirmation that the goal is to prohibit an encoding prefix as opposed to the current behavior that ignores an encoding prefix.
- Corentin replied affirmatively.
- JeanHeyd noted that this does create an incompatibility with C then, but it probably isn't a big deal.
- Tom asked if Corentin's code survey accounted for string literals produced by macro expansion.
- Corentin replied that it did not.
- Jens noted that a macro expansion could produce a string literal with an encoding prefix.
- PBrett observed that making the presence of an encoding prefix ill-formed doesn't mean an implementation has to reject the code; it just means that a diagnostic is required.
- Steve stated that the intent of `_Pragma` is to be an alternative to `#pragma`, one that is friendly to macros, but there is no encoding involved.
- Jens agreed; no encoding involved, an encoding prefix serves no purpose.
- Jens noted that `_Pragma` is relatively new; it was introduced in C99.
- JeanHeyd observed that an `_Pragma` expression written with a wide string literal might show up on Windows due to use of a `TCHAR` aware macro.
- JeanHeyd suggested that it might be best to just follow C; but that either all encoding prefixes should be allowed and ignored, or they should all be disallowed.
- Corentin stated that programmers don't tend to use a macro with `_Pragma`.
- Tom disagreed and noted that `_Pragma` was introduced as a macro friendly alternative to `#pragma`.
- Tom then reverted his disagreement by noting that macros can be used with `#pragma` as well (so long as the `#pragma` tokens themselves are not the result of macro expansion).
- Mark asked if the grammar for `_Pragma` should be specified using *string-literal*.
- Jens replied that that is not an SG16 concern.
- Proposal 11: Make character literals in preprocessor conditional behave like they do in C++ expression
 - Corentin explained that character literal values can be inspected in preprocessor conditional directives during translation phase 4, but the values observed then are not required to match

observations for character literal values during translation phase 7.

- Corentin stated that the existing specification is presumably intended to support an external preprocessor.
- Corentin added that the intent is to reduce the number of implementation-defined encodings in the standard and to match existing practice and existing programmer expectations as determined by code surveys.
- Hubert noted that the example is incorrect assuming the intent was to compare against ASCII values; the `\x65` and `0x65` should presumably be `\x41` and `0x41` respectively.
- Hubert confirmed that compilers on z/OS use the same character encoding for character literal observations made during translation phase 4 and translation phase 7.
- Tom asked about cross compilers; a tool chain that uses an external preprocessor may not have support for, or be aware of, the character encoding observed at translation phase 7.
- Hubert responded that, in cross compilation scenarios, headers are highly likely to be consistent between a cross compilation environment and native environment on the target; the observed values therefore need to be consistent in both environments.
- Steve agreed; many cross compilation environments require mounting a remote filesystem for access to headers and libraries.
- Tom stated that there are two possibilities for the character encoding observed at translation phase 4; either the internal encoding or the execution encoding.
- PBrett noted that the internal encoding should never be observable.
- Tom stated that this is technically a breaking change.
- Jens agreed, but noted that we know of no implementations that would be broken.
- Jens added that it would be odd to associate a character encoding with the preprocessor.
- Jens stated that, from a wording perspective, we'll need to state that the preprocessor must perform the same conversion for character literals at translation phase 4 that is done at translation phase 5.
- PBrett stated that he had been unaware that the preprocessor was potentially using a distinct character encoding; that would likely be a surprise to many programmers.
- Tom noted that this potentially has implementation impact since compiler drivers will need to coordinate with the preprocessor and the compiler to ensure a matching character encoding is used.
- Steve noted a typo; in the third paragraph, "where" should be "were" in "Of the 50 usages of the pattern, all but one where in C libraries."
- Proposal 12: Phase 6 needs fixing
 - Corentin expressed uncertainty regarding how to address this issue.
 - Corentin opined that it is odd that the encoding would not be determined by the first string literal.
 - Corentin stated that, if a time machine were to suddenly materialize, the standard would require the encoding-prefix to be present for the first string literal. But it is likely too late to make such a change now.
 - Corentin added that this issue will be less significant if Jens' [P2201](#) is adopted.
 - Jens mentioned that a [D2201R1](#) now exists with the EWG requested changes.
 - Jens added that P2201 isn't fundamentally related to this issue, though.
 - Jens stated that [core issue 2455](#) now tracks this issue.
 - Jens directed the group to [a draft paper](#) that demonstrates one way to address this issue.
 - Jens opined that this issue is really just a core issue; the wording is defective, but the intent is clear in [\[5.13.5\]](#).
 - Tom agreed.
 - Steve reminded the group that there is implementation divergence.

- Polls on [P2178R1](#) proposals:

- Proposal 2: What is a whitespace or a new-line?
 - Hubert stated that this proposal deals in the formation and replacement of newlines and therefore can not be meaningfully separated from the noted core issue; [core issue 1655](#).
 - Corentin responded that the intent is that line endings are preserved through translation phase 1.
 - Tom noted that specifying that intent is difficult since translation phase 1 is so loose.
 - Corentin suggested that a new grammar term for newline may be needed.
 - PBrett stated that the current poll should focus on whether we support the proposed direction.
 - Hubert asserted that an implementation survey should be done since line numbers are observable via `__LINE__` and `std::source_location`.
 - Hubert added that this proposal introduces challenges for compilers that open source files as "text" files since doing so transparently mutates line endings.
 - Jens asserted that a wording direction that would suffice as a proposed resolution for [core issue 1655](#) is needed before polling.
 - Hubert raised concerns about implementations that read source code from datasets with fixed length records.
 - Tom asked if anyone had a fundamental objection to the general direction.
 - No objections were raised.
- Proposal 3: Preserve Normalization forms
 - Jens asserted that this proposal needs to address how to tunnel code points through translation phase 1 and translation phase 5.
 - Hubert noted that an implementation would have to define how it determines whether a source file is Unicode encoded.
 - Hubert asked what it means to preserve normalization through translation phase 5 if the execution character set is not Unicode.
 - Corentin replied that the intent is that code point sequences that contain combining characters cannot be composed during translation phase 5.
 - **Poll: Proposal 3: We agree that, for Unicode source files, that normalization is preserved through translation phases 1 and 5.**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
- Proposal 4: Making trailing whitespaces non-significant
 - Tom declared that this is not an SG16 concern and that Corentin is free to take this directly to EWG.
- Proposal 5: Restricting multi-characters literals to members of the Basic Latin Block
 - Tom suggested that the restriction be redefined in terms of characters that are encodable as a single code unit since some characters in this block may not be encodable or may not be encodable as a single code unit.
 - Corentin expressed concern about portability.
 - PBrett suggest changing the restriction to the basic source character set.
 - **Poll: Proposal 5: We support this direction modified in terms of the basic source character set.**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
- Proposal 6: Making wide characters literals containing multiple or unrepresentable c-char ill-formed
 - **Poll: Proposal 6: We support making wide multicharacter literals ill-formed.**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
 - **Poll: Proposal 6: We support making wide non-encodable character literals ill-formed.**
 - **Attendees: 10**

- **No objection to unanimous consent.**
- Proposal 7: Making conversion of character and string literals to execution and wide execution encoding ill-formed for unrepresentable c-char
 - Steve asked if a source file containing Unicode 13 characters would be ill-formed if compiled by a compiler that only supports Unicode 12.
 - PBrett asked for confirmation that a sparkle emoji present in a ordinary string literal in a Unicode encoded source code would be ill-formed if the execution character set is ISO-8859-1.
 - Corentin replied that it would be.
 - Jens stated that this restriction could always be worked around by defining ones own execution character set, so this doesn't provide much benefit.
 - Hubert agreed that the normative impact is dubious.
 - Jens suggested that polling be postponed since there are concerns that appear to warrant additional discussion.
 - Tom agreed.
- Proposal 8: Enforcing the formation of universal escape sequences in phase 2 and 4
 - Tom declared that this is not an SG16 concern and that Corentin is free to take this directly to EWG.
- Proposal 10: Make L in `_Pragma` ill-formed
 - **Poll: Proposal 10: We agree to make all encoding-prefixes in `_Pragma` ill-formed.**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
- Proposal 11: Make character literals in preprocessor conditional behave like they do in C++ expression
 - Hubert asserted that opinions on this should be gathered from WG14.
 - **Poll: Proposal 11: We agree that the same character encoding should be used for character literal in translation phase 4 and 7.**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
- Proposal 12: Improved wording for phase 6 string concatenation
 - Tom declared that this is not an SG16 concern.
- Tom stated that the next telecon will be held on September 9th.